

Licence MIASHS première année

Rapport de projet informatique

MindChallenger

Projet réalisé du 7 avril 2026 au 23 mai 2026

Membres du groupe

Lyna Lamloum — N° étudiant : 45006183

Océan Hivet — N° étudiant : 45017865

Wiam Achahboun — N° étudiant : 45006152

Remerciements

Nous remercions notre enseignant pour ce cours, qui nous a permis de réaliser un projet plus libre et créatif qu'un projet de programmation classique.

Ce projet nous a aussi poussés à découvrir de nouvelles façons de travailler, notamment autour des outils d'IA générative, du développement web et de la gestion d'un projet plus ambitieux que ce que nous avons fait jusque-là.

Enfin, merci aux personnes ayant testé le projet et nous ayant fait des retours pendant sa création.

Table des matières

1	Introduction	5
2	Environnement de travail	5
2.1	Outils et logiciels	5
2.2	Organisation du groupe	6
2.3	Utilisation de l'IA dans le développement	6
3	Description du projet et objectifs	6
3.1	L'univers narratif	6
3.2	Les personnages	7
3.3	Les modes de jeu	7
3.4	Les chapitres du mode Histoire	7
4	Bibliothèques, Outils et technologies	7
4.1	Choix d'architecture : zéro dépendance	8
4.2	Le serveur Node.js (server.js)	8
4.3	L'API Anthropic (Claude)	9
4.4	Les routes API du serveur	10
4.5	Le registre de sources vérifiées	10
4.6	L'interface frontend (app.js et styles.css)	10
4.7	Fonctionnalités détaillées	11
4.7.1	Le système de débat	11
4.7.2	Les quatre niveaux de difficulté	11
4.7.3	Le verdict et la fiche de progression	11
4.7.4	Le ticker de blagues	11
4.7.5	Le Carnet de Vesper et ses pages débloquables	12
4.7.6	Le Grimoire	12
4.7.7	Le Carnet de débat	12
4.7.8	Sauvegarde automatique et persistance	12
4.7.9	Lancement en un clic sous Windows	12
4.7.10	Réinitialisation de la progression	13
5	Travail réalisé	13
5.1	Fonctionnalités réalisées	13
5.2	Fonctionnalités non réalisées	13
5.3	Répartition réelle du travail	14
6	Difficultés rencontrées	14
6.1	Faire produire du JSON valide à l'IA	14
6.2	Calibrer les personnages	14
6.3	Sécuriser la clé API	15
6.4	La gestion asynchrone	15
6.5	Intégration de plusieurs fournisseurs d'IA	15
6.6	Compatibilité des navigateurs	15

7	Bilan	15
7.1	Limites du projet	16
7.2	Conclusion	16
7.3	Perspectives	17
8	Bibliographie	18
9	Webographie	19
10	Annexes	20
A	Cahier des charges	20
A.1	Architecture complète du projet	20
B	Exemple d'exécution du projet	21
C	Manuel utilisateur	30

1 Introduction

MindChallenger est un jeu de débat narratif que l'on a développé en première année de licence MIAHS. L'idée de départ était simple : et si on pouvait s'entraîner à argumenter face à un adversaire artificiel, dans un univers fictif qui rende l'exercice plus engageant qu'un simple chatbot ?

Nous avons donc créé la **Cour d'Aster**, un tribunal intellectuel fictif où le joueur incarne un Aspirant-Orateur qui choisit un adversaire, défend une position sur un sujet philosophique ou de société, et reçoit le verdict d'une juge à la fin. Le tout est propulsé par l'API Claude d'Anthropic, qui joue à la fois l'adversaire, la juge, et le coach de fin de partie.

Notre objectif principal était de construire un projet où l'IA ne soit pas juste un ajout secondaire, mais une vraie partie du gameplay. Une grande partie du code a quand même été développée directement par le groupe : l'interface, la narration, la gestion des données et le serveur.

Lien vers le dépôt GitHub public

<https://github.com/lynalamloun/projet-informatique-L1>

2 Environnement de travail

2.1 Outils et logiciels

Nous avons travaillé sous Windows 10 et 11. Le tableau suivant récapitule les outils utilisés :

Outil	Utilisation dans le projet
Node.js v20+	Serveur local, moteur d'exécution JavaScript
Visual Studio Code	Éditeur principal (avec les extensions ESLint et Prettier)
Google Drive	Collaboration et partage de fichiers en temps réel
Git + GitHub	Versionnement du code, dépôt public
Claude (Anthropic)	Assistant IA : aide à la programmation, rédaction des prompts
ChatGPT (OpenAI)	Génération des portraits et fonds de scène (images IA), aide ponctuelle sur le CSS
Overleaf	Rédaction du rapport en $\text{\LaTeX}$
Chrome DevTools	Débogage de l'interface, inspection du réseau

TABLE 1 – Outils utilisés pendant le projet

2.2 Organisation du groupe

Nous avons organisé le travail via Google Drive, qui nous a permis de partager et modifier les fichiers en temps réel. Nous nous retrouvions régulièrement, en présentiel ou en visioconférence, pour faire le point sur les avancées et résoudre les blocages ensemble. Le code a ensuite été publié sur GitHub pour satisfaire l'exigence d'un dépôt public.

2.3 Utilisation de l'IA dans le développement

C'est sur ce point que le projet se distingue d'un projet de programmation classique. On a utilisé principalement Claude comme assistant de développement, et voici comment s'est articulée cette collaboration :

- **Développé sans assistance IA** : la structure HTML de base (`index.html`), le routing entre les écrans (la fonction `screen()` dans `app.js`), la logique de sauvegarde en `localStorage`, le script de lancement Windows (`.bat`), l'organisation des dossiers, la gestion du `.gitignore`, et la totalité de la narration (personnages, univers, dialogues, le carnet de Vesper).
- **Développé avec l'assistance de Claude** : la partie serveur Node.js et notamment la couche d'appels à l'API Anthropic, les prompts système des personnages, l'algorithme de sélection des sources, le système de validation JSON des réponses IA.
- **Généré puis adapté** : ChatGPT a généré les portraits des personnages et les fonds de scène via DALL-E. On a retravaillé certains styles CSS soi-même pour coller à l'ambiance voulue.

Chaque suggestion de code produite par l'IA a été relue, testée et souvent modifiée, soit parce qu'elle ne correspondait pas exactement au besoin, soit parce qu'il était nécessaire de comprendre ce qu'elle faisait avant de l'intégrer. Il est difficile de quantifier précisément la répartition, mais nous estimons avoir rédigé ou réécrit une part significative du code.

3 Description du projet et objectifs

3.1 L'univers narratif

L'histoire se déroule dans la Cour d'Aster, un tribunal intellectuel fictif. Le joueur arrive à Aster par le train, découvre une immense statue brisée dans le hall : celle de **Vesper**, le plus brillant Orateur de l'histoire, disparu l'An 7 sans explication. La juge Éléonore Moreau l'accueille et l'inscrit aux Épreuves.

Le fil narratif repose sur le carnet de Vesper, qui se déverrouille en intégralité une fois le mode Histoire terminé. Cette promesse d'un secret à découvrir donne envie de progresser au-delà du premier débat.

3.2 Les personnages

Quatre adversaires ont été conçus avec des personnalités très différentes, ce qui change vraiment l'expérience de jeu :

Personnage	Rôle dans l'histoire	Style de débat
Marc Varenne	Ancien historien de la Cour	Cynique, direct, exige des preuves concrètes et des chiffres. Attaque les effets pervers.
Nora Belkacem	Oratrice du port d'Aster	Intense, morale, combative. Humanise le débat, nomme les gens affectés.
Adrienne Valmont	Directrice des Épreuves	Froide, précise. Attaque les définitions floues et les pré-supposés.
Socrate	Dernier témoin de Vesper	Ne donne jamais son avis. Pose uniquement des questions courtes et déstabilisantes.

TABLE 2 – Les quatre adversaires du jeu

Il y a aussi trois avatars jouables (Aïssata Ndiaye, Sebastian Kovács, Yunseo Park), chacun avec un style présenté dans un écran de sélection dédié.

3.3 Les modes de jeu

- **Débat libre** : le joueur choisit son sujet, son adversaire, son camp (pour/contre) et la difficulté.
- **Sujet surprise** : l'IA génère 3 sujets aléatoires adaptés au thème et à la difficulté choisis, le joueur en sélectionne un. Il est possible de régénérer de nouveaux sujets à volonté.
- **Mode Histoire** : quatre chapitres narratifs avec un adversaire et un sujet imposés, plus un épilogue.

3.4 Les chapitres du mode Histoire

4 Bibliothèques, Outils et technologies

Cette section détaille les choix techniques du projet : l'architecture du serveur, l'intégration de l'API Claude, et l'ensemble des fonctionnalités implémentées.

Chapitre	Adversaire	Question philosophique
Prologue	—	Arrivée à Aster, découverte de la statue de Vesper
I	Nora Belkacem	Faut-il toujours dire la vérité, même lorsqu'elle fait souffrir ? (réf. Antigone)
II	Marc Varenne	Faut-il laisser une trace de son existence ? (réf. L'Écclésiaste, Camus)
III	Adrienne Valmont	Les humains sont-ils vraiment capables d'être libres ? (réf. Rousseau, Kant)
IV	Socrate	Faut-il toujours remettre ses certitudes en question ? (réf. Nietzsche, Kierkegaard)
Épilogue	Juge Moreau	Le carnet de Vesper, le verdict final, la vraie leçon

TABLE 3 – Chapitres du mode Histoire

4.1 Choix d'architecture : zéro dépendance

L'un des choix techniques dont nous sommes les plus satisfaits est d'avoir construit **tout le projet sans aucune dépendance npm**. Sans React, sans Express, sans axios. Le serveur utilise uniquement les modules natifs Node.js (`http`, `fs`, `path`, `url`) et le frontend est du JavaScript vanilla pur.

Les avantages sont concrets : aucun dossier `node_modules` à installer, aucune dépendance susceptible de rompre la compatibilité. Le projet se lance avec un simple `node server.js`.

Pour comprendre la création d'un serveur HTTP en Node.js natif, nous avons consulté la documentation officielle [NODEJS] et ce tutoriel [TUTONODEJS].

4.2 Le serveur Node.js (server.js)

Le serveur fait environ 400 lignes et remplit deux rôles distincts :

1. **Serveur de fichiers statiques** : il sert l'HTML, le CSS, le JavaScript et les images du dossier `public/` vers le navigateur.
2. **Proxy API sécurisé** : il reçoit les appels depuis l'interface et les transmet à l'API Claude d'Anthropic, sans jamais exposer la clé API au navigateur.

La partie que nous avons développée en priorité, avec l'appui de la documentation officielle, comprend la fonction de routage (`switch` sur les URL), le chargement du fichier `.env.local`, et le serveur de fichiers statiques avec la gestion des types MIME. La partie développée avec l'assistance de Claude regroupe les fonctions d'appel à l'API Anthropic et les prompts système.

```

1 function loadEnv() {
2   for (const filename of [".env.local", ".env"]) {
3     try {
4       const raw = require("node:fs").readFileSync(
5         path.join(ROOT, filename), "utf8"
6       );
7       for (const line of raw.split(/\r?\n/)) {
8         const t = line.trim();

```

```
9         if (!t || t.startsWith("#")) continue;
10        const idx = t.indexOf("=");
11        if (idx === -1) continue;
12        const key = t.slice(0, idx).trim();
13        const value = t.slice(idx+1).trim()
14            .replace(/^[\"']|['\"]$/g, "");
15        if (key) process.env[key] = value;
16    }
17    } catch { /* fichier absent, on continue */ }
18 }
19 }
```

Listing 1 – Chargement des variables d’environnement (server.js — écrit sans IA)

Pour la gestion des routes HTTP, nous nous sommes appuyés sur la documentation officielle Node.js [NODEJS] et de ce guide sur les API REST [APIREST].

4.3 L’API Anthropic (Claude)

C’est le cœur du projet. On appelle l’API Claude d’Anthropic via des requêtes HTTP POST. Le modèle utilisé est `claude-sonnet-4-5`. Chaque appel contient un *system prompt* (qui définit le comportement du personnage) et un message utilisateur (le contenu du débat). L’API répond en JSON.

Pour obtenir une clé API Anthropic et comprendre comment l’utiliser, on a suivi la documentation officielle [?] et ce tutoriel d’introduction [TUTOAPI].

```
1  async function claude(systemPrompt, userContent) {
2      const res = await fetch("https://api.anthropic.com/v1/
3          messages", {
4          method: "POST",
5          headers: {
6              "x-api-key": ANTHROPIC_KEY,
7              "anthropic-version": "2023-06-01",
8              "Content-Type": "application/json",
9          },
10         body: JSON.stringify({
11             model: "claude-sonnet-4-5",
12             max_tokens: 2048,
13             system: systemPrompt,
14             messages: [{ role: "user", content: userContent }],
15         }),
16     });
17     const data = await res.json();
18     const text = data.content?.[0]?.text || "";
19     return parseJson(text); // nettoie les backticks Markdown
20     eventuels
21 }
```

Listing 2 – Appel à l’API Claude (server.js — développé avec Claude)

4.4 Les routes API du serveur

Route	Rôle
POST /api/debate/respond	Génère la réponse de l'adversaire IA
POST /api/debate/verdict	La juge Moreau analyse le débat et rend son verdict
POST /api/debate/final-report	Génère la fiche de progression personnalisée
POST /api/debate/generate-topics	Génère 3 sujets de débat selon le thème et la difficulté
POST /api/debate/search-sources	Retourne les sources pertinentes pour un sujet donné
POST /api/config	Permet de mettre à jour la clé API depuis l'interface
GET /api/status	Indique si le serveur et l'IA sont actifs

TABLE 4 – Routes exposées par le serveur

4.5 Le registre de sources vérifiées

Pour éviter que l'IA invente des références (ce qu'elle fait naturellement si on ne l'encadre pas), on a intégré dans `server.js` un registre de **20 sources réelles et vérifiées** : OCDE, OMS, UNESCO, GIEC, Stanford HAI, Pew Research Center, Reuters Institute, etc.

Avant chaque appel à l'API, un algorithme de correspondance par mots-clés sélectionne les sources les plus pertinentes par rapport au sujet du débat, et les injecte dans le prompt en précisant : ■ *tu ne peux citer que celles-ci* ■. L'IA ne peut donc pas inventer de sources.

4.6 L'interface frontend (app.js et styles.css)

Tout le frontend est en **JavaScript vanilla** (sans framework), environ 1 339 lignes. Ce choix était délibéré : nous souhaitions comprendre chaque ligne de code produite, plutôt que de s'appuyer sur un framework dont nous n'aurions pas maîtrisé les mécanismes internes.

La logique centrale fonctionne comme un petit moteur de rendu maison : un objet `state` contient l'état courant du jeu (écran actif, personnage choisi, messages du débat...), et une fonction `render()` reconstruit l'interface à chaque changement. C'est le principe des frameworks modernes, appliqué à la main.

Le CSS (1 362 lignes) utilise des variables CSS pour les couleurs et les accents de chaque personnage, ce qui permet de changer toute la palette du jeu en une ligne.

Pour comprendre comment structurer une SPA sans framework, on a consulté ce tutoriel [\[SPA\]](#).

4.7 Fonctionnalités détaillées

4.7.1 Le système de débat

Au cœur du jeu, il y a le débat. Le joueur saisit un argument dans une zone de texte, le serveur l'envoie à l'API Claude avec tout le contexte (sujet, personnage, difficulté, historique des échanges), et l'IA renvoie un JSON structuré contenant :

- **reply** : la réponse argumentée du personnage (au moins 3 phrases construites) ;
- **questionToUser** : une question finale qui relance le débat ;
- **strategyUsed** : la stratégie rhétorique utilisée (utile pour la fiche de progression) ;
- **sources** : les sources du registre éventuellement citées ;
- **memoryNotes** : des éléments que l'IA retient pour la suite du débat.

4.7.2 Les quatre niveaux de difficulté

Les niveaux modifient concrètement le comportement de l'IA en ajoutant des instructions spécifiques dans le prompt système :

- **Facile** : l'adversaire est bienveillant, ses objections sont simples, il concède facilement (*Tu as tout à fait raison sur ce point.*). Idéal pour commencer.
- **Moyen** : débat sérieux, l'adversaire repère les imprécisions mais reste accessible.
- **Difficile** : l'adversaire attaque la logique interne, revient sur les contradictions, cite des sources.
- **Expert** : l'adversaire exploite l'historique du débat pour pointer les glissements de position, impose des dilemmes vrais, ne laisse passer aucune généralité.

4.7.3 Le verdict et la fiche de progression

À la fin du débat, la juge Éléonore Moreau analyse l'intégralité des échanges et rend un verdict sur 8 critères : clarté de la thèse, qualité des définitions, logique, preuves, réponses aux objections, nuance, persuasion, usage des sources. Elle attribue un score à chaque camp.

En parallèle, un **rapport de coaching personnalisé** est généré : il contient les points forts du joueur, ses faiblesses, des exemples de réponses idéales, les arguments qu'il n'a pas pensé à utiliser, du vocabulaire rhétorique utile et des phrases types. Aucun conseil générique n'est autorisé par le prompt : tout doit être ancré dans ce qui s'est dit dans ce débat précis.

Note pour le testeur : le calcul du verdict et du rapport de coaching nécessite deux appels successifs à l'API Claude. Ce double traitement peut prendre entre 15 et 30 secondes ; un indicateur visuel de chargement s'affiche pendant ce délai. Il s'agit d'un comportement normal lié au temps de réponse de l'IA, et non d'un dysfonctionnement.

4.7.4 Le ticker de blagues

En haut de l'interface, un bandeau de texte défilant affiche toutes les sept secondes une citation humoristique à thème philosophique ou sociétal. Cette petite touche d'humour, entièrement rédigée par nos soins, vise à alléger l'atmosphère sans interrompre le jeu. Le *ticker* est géré en JavaScript pur, avec une animation de fondu par transition d'opacité CSS pilotée par `setInterval`.

4.7.5 Le Carnet de Vesper et ses pages débloquentables

Le Carnet de Vesper est une section narrative qui se déverrouille en intégralité à l'issue du mode Histoire. Une fois l'épilogue complété, le joueur accède aux six pages du carnet : les premières sont des notes de débat structurées, rédigées dans la voix de Vesper après chaque épreuve ; les suivantes deviennent plus courtes et fragmentées, reflétant l'évolution intérieure du personnage jusqu'à sa dernière phrase.

La sixième et dernière page, intitulée *Page Déclassée*, contient un petit *easter egg* à l'attention du correcteur. Nous préférons ne pas en révéler la teneur ici. Saurez-vous le trouver ?

4.7.6 Le Grimoire

Le Grimoire est un manuel du débatteur intégré directement dans le jeu, consultable à tout moment depuis le menu. Il contient plusieurs sections pédagogiques :

- Comment construire un argument solide ;
- Les techniques de réfutation ;
- Les biais cognitifs courants (et comment les exploiter ou les éviter) ;
- Un vocabulaire rhétorique avec des exemples ;
- Des phrases types pour ouvrir, contester, concéder et conclure.

4.7.7 Le Carnet de débat

Accessible depuis la barre de navigation, le Carnet de débat rassemble toutes les données de progression du joueur (sauvegardées en `localStorage`). Il présente :

- Des **statistiques globales** : nombre de débats joués, victoires, défaites, nuls.
- Un **suivi de compétences** : cinq axes évalués à chaque débat (logique, sources, réfutation, nuance, persuasion), agrégés sous forme de barres de progression.
- Les **Fragments de Vesper** : les citations clés associées à chaque chapitre complété, affichées comme des jalons narratifs.
- L'**historique des débats** : sujet, adversaire, résultat et fiche d'amélioration consultable pour chaque partie.

4.7.8 Sauvegarde automatique et persistance

Toute la progression du joueur est sauvegardée automatiquement dans le `localStorage` du navigateur : avatar choisi, nom, chapitres débloquentés et complétés, historique des débats, pages du Carnet de Vesper déverrouillées. La sauvegarde se fait à chaque action importante, sans que le joueur ait à faire quoi que ce soit.

4.7.9 Lancement en un clic sous Windows

Nous avons rédigé un script `.bat` permettant à tout utilisateur de lancer l'application sans connaître Node.js. Le script :

1. vérifie que Node.js est installé (et affiche un message d'erreur clair sinon) ;
2. demande la clé API Anthropic si le fichier `.env.local` est absent ;
3. sauvegarde la clé dans `.env.local` ;

4. lance `node server.js` ;
5. ouvre automatiquement `http://localhost:5173` dans le navigateur par défaut.

Ce script a été entièrement rédigé par nos soins, en nous aidant de la documentation Microsoft sur les scripts batch [\[BAT\]](#).

4.7.10 Réinitialisation de la progression

Le bouton *Nouvelle Traversée*, accessible depuis la barre de navigation, permet de remettre la progression à zéro en deux étapes de confirmation. La première demande si l'on souhaite repartir du prologue ; la seconde, intitulée *Effacer définitivement cette partie ?*, liste précisément ce qui sera supprimé (personnage, chapitres, débats, fiches, statistiques, Carnet de Vesper) et précise que les clés API ne sont pas affectées. Ce double-verrou évite les suppressions accidentelles.

5 Travail réalisé

5.1 Fonctionnalités réalisées

Tout ce qu'on avait prévu au départ a été réalisé. Voici un récapitulatif :

Fonctionnalité	Réalisée ?
Système de débat IA avec 4 personnages distincts	Oui
4 niveaux de difficulté	Oui
3 modes de jeu (libre, surprise, histoire)	Oui
Mode Histoire complet (4 chapitres + épilogue)	Oui
Verdict argumenté de la juge Moreau	Oui
Fiche de coaching personnalisée	Oui
Registre de 20 sources vérifiées	Oui
Carnet de Vesper (pages débloquentables)	Oui
Ticker de blagues	Oui
Easter egg (dernière page du Carnet de Vesper)	Oui
Grimoire (manuel du débateur)	Oui
Carnet de débat (historique + stats)	Oui
Sauvegarde automatique en localStorage	Oui
Lancement en un clic (script .bat Windows)	Oui
Design complet (typographie, animations, portraits IA)	Oui
Sécurité de la clé API (côté serveur uniquement)	Oui

TABLE 5 – Récapitulatif des fonctionnalités réalisées

5.2 Fonctionnalités non réalisées

Quelques idées qu'on avait en tête mais qu'on n'a pas eu le temps d'implémenter :

- **Mode multijoueur** : deux joueurs humains s'affrontent, l'IA joue la juge. C'est techniquement faisable mais ça aurait demandé une gestion de sessions côté serveur qu'on n'avait pas le temps d'apprendre.

- **Export PDF de la fiche de progression** : on a cherché des solutions côté navigateur (`window.print()`, librairies JS), mais la mise en forme n'était pas satisfaisante.
- **Sons et musique d'ambiance** : on avait prévu une ambiance sonore de tribunal, mais l'intégration audio s'est révélée plus compliquée que prévu avec une SPA.

5.3 Répartition réelle du travail

Membre	Contributions principales
Lyna Lamoum	Serveur Node.js, intégration API Anthropic, algorithme de sélection des sources, routes API, fichiers <code>.env</code>
Océan Hivet	Interface frontend (<code>app.js</code>) : navigation, écrans, débat, verdict, carnet, grimoire, sauvegarde localStorage
Wiam Achahboun	Univers narratif, prompts des personnages, CSS complet, génération et intégration des assets visuels, script <code>.bat</code>

TABLE 6 – Répartition réelle du travail

6 Difficultés rencontrées

6.1 Faire produire du JSON valide à l'IA

Il s'agissait sans doute de la difficulté la plus inattendue. Nous pensions qu'indiquer à Claude de répondre en JSON suffirait. En pratique, l'IA entourait fréquemment sa réponse de balises Markdown (triples backticks json), ou construisait un JSON avec des virgules manquantes, ou ajoutait du texte avant l'accolade ouvrante.

Il a fallu ajouter une fonction `parseJson()` côté serveur pour nettoyer le texte avant l'analyse, et affiner les prompts en y incluant des exemples concrets du format attendu (*few-shot prompting*). Ce processus d'itération a mobilisé plusieurs jours de travail.

6.2 Calibrer les personnages

Dans les premières versions, les adversaires avaient tendance à se ressembler. Marc Varenne était parfois aussi philosophique qu'Adrienne Valmont. Socrate faisait des discours au lieu de poser des questions. Nous avons consacré un temps considérable à tester et ajuster les prompts pour que chaque personnage soit vraiment distinct et reconnaissable.

Le cas de Socrate était particulièrement délicat : on voulait qu'il pose *uniquement* des questions, sans jamais donner son avis. Il a été nécessaire d'ajouter dans son prompt une règle en majuscules : `TU NE DONNES JAMAIS TON OPINION. TU POSES UNIQUEMENT DES QUESTIONS.`

6.3 Sécuriser la clé API

Notre première approche consistait à placer la clé API directement dans le code JavaScript côté client, ce qui aurait représenté un risque de sécurité sérieux : toute personne ouvrant les outils de développement du navigateur aurait pu l'extraire. La lecture de ressources sur les bonnes pratiques de sécurité API nous a conduits vers la notion de variables d'environnement et de serveur proxy [APISEC], et c'est ce qui nous a conduits à créer `server.js`.

6.4 La gestion asynchrone

La programmation asynchrone JavaScript (`async/await`, Promises) était nouvelle pour une partie du groupe. Des erreurs telles que `Cannot read properties of undefined`, ou des résultats indéfinis liés à l'oubli d'un `await`, ont nécessité plusieurs séances de débogage. La documentation MDN [MDNASYNC] a été une référence essentielle pour débloquer ces situations.

6.5 Intégration de plusieurs fournisseurs d'IA

L'une des difficultés les plus chronophages a été de trouver le bon fournisseur d'API. Nous avons commencé par tenter d'intégrer plusieurs alternatives avant de nous stabiliser sur Claude.

Nous avons d'abord essayé **OpenRouter**, qui permet d'accéder à plusieurs modèles via une seule API. Le problème : les réponses variaient fortement d'un modèle à l'autre, et certains modèles gratuits refusaient régulièrement de répondre en JSON structuré, rendant le parsing impossible. Nous avons ensuite testé **Google Gemini**, dont la documentation était plus difficile à prendre en main et dont les réponses, pour les prompts de personnages, manquaient de cohérence narrative. Nous avons également essayé **Mistral**, dont les réponses étaient trop courtes pour les débats en niveau difficile et expert.

Ces tentatives successives nous ont appris à lire les documentations d'API, à comparer des comportements de modèles, et à comprendre pourquoi un prompt bien calibré sur un modèle peut complètement échouer sur un autre. Finalement, Claude s'est imposé pour la qualité de ses réponses structurées et la cohérence de ses personnages.

Le frontend conserve d'ailleurs une trace de ces essais : `app.js` contient encore des constantes pour Gemini et OpenRouter (clé de stockage, modèle par défaut), qui auraient permis au joueur de choisir son fournisseur. Faute de temps pour tester rigoureusement toutes les combinaisons, nous avons finalement verrouillé l'application sur Claude uniquement.

6.6 Compatibilité des navigateurs

Certaines propriétés CSS (notamment les animations sur les pseudo-éléments et les variables CSS dans certains contextes) se comportaient différemment entre Chrome, Firefox et Edge. On a résolu la plupart des problèmes avec des règles CSS plus explicites, mais quelques petites différences visuelles subsistent.

7 Bilan

7.1 Limites du projet

MindChallenger repose en grande partie sur un service externe, ce qui entraîne plusieurs contraintes dont nous avons pris conscience au fil du développement.

La première est la **dépendance à l'API Anthropic**. Sans clé valide et sans connexion internet, le jeu ne fonctionne pas : l'adversaire, la juge et les rapports de coaching sont entièrement générés en ligne. Une interruption du service Anthropic suffit à rendre l'application inutilisable.

La deuxième limite concerne le **coût des appels IA**. Chaque échange, chaque verdict et chaque rapport consomme des tokens facturés à la consommation. Pour un usage intensif ou un déploiement public, cela représente un coût réel qui n'a pas été budgété dans ce projet.

Troisièmement, comme tout modèle de langage, Claude peut produire des **hallucinations** : formuler un argument incorrect mais convaincant, ou légèrement déformer le contenu d'une source. Nous avons réduit ce risque en imposant un registre de sources fermé et en validant le JSON côté serveur, mais aucune garantie absolue n'est possible.

Le **temps de réponse** constitue également une limite perceptible. Chaque appel à l'API prend entre 3 et 15 secondes selon la longueur attendue, et le verdict nécessite deux appels successifs. Ce délai peut nuire à la fluidité du jeu, en particulier en niveau Expert.

L'IA n'a par ailleurs **pas de mémoire longue** : seul l'historique récent du débat est transmis à chaque appel. Sur des sessions très longues, les premiers échanges sortent du contexte envoyé, ce qui peut amener l'adversaire à ignorer des arguments formulés en début de débat.

Enfin, **évaluer objectivement un débat** reste une tâche difficile même pour un humain. Les critères de la juge (logique, nuance, persuasion...) sont subjectifs par nature, et le verdict produit peut parfois sembler discutable. Il s'agit davantage d'un retour formatif que d'une évaluation rigoureuse.

7.2 Conclusion

Ce projet nous a demandé plus de travail qu'anticipé, surtout sur des aspects qu'on n'aurait pas imaginé aussi difficiles au départ. La gestion des appels asynchrones, par exemple, nous a causé beaucoup d'erreurs avant qu'on comprenne vraiment comment fonctionnaient les *Promises* en JavaScript. Les essais avec plusieurs fournisseurs d'IA ont aussi été chronophages, même si finalement instructifs.

Si c'était à refaire, on prendrait plus de temps en amont pour tester les outils avant de commencer à coder. On a perdu pas mal de temps à intégrer Gemini ou OpenRouter pour finalement revenir à Claude, et la même chose s'est produite avec certains choix CSS qu'on a dû entièrement reprendre.

Ce qui nous a le plus surpris, c'est à quel point l'écriture des prompts est un travail à part entière. On pensait que ce serait la partie la plus rapide ; ça a finalement été l'une des plus longues, et des plus intéressantes.

Nous sommes satisfaits du résultat et espérons que MindChallenger sera agréable à tester.

7.3 Perspectives

Dans l'hypothèse d'une poursuite du développement, les axes prioritaires seraient :

- **Nouveaux personnages** : un scientifique, un économiste, un politique. Chacun avec un domaine de sources dédié.
- **Mode multijoueur** : deux humains débattent, l'IA joue la juge.
- **Déploiement en ligne** : héberger l'application sur un service comme Railway ou Render pour que n'importe qui puisse y accéder sans installer Node.js.
- **Export de la fiche** : générer un PDF propre de la fiche de progression.
- **Internationalisation** : version anglaise des débats pour pratiquer l'argumentation dans une autre langue.

8 Bibliographie

- [ANT] Anthropic, *Claude API Documentation — Messages endpoint*, 2025.
<https://docs.anthropic.com/en/api/messages>
- [ANT2] Anthropic, *Introduction to Claude — Anthropic Documentation*, 2025.
<https://docs.anthropic.com/en/docs/intro-to-claude>
- [ANT3] Anthropic, *Models Overview — Claude claude-sonnet-4-5*, 2025.
<https://docs.anthropic.com/en/docs/about-claude/models/overview>
- [GEM] Google, *Gemini API Documentation*, 2024.
<https://ai.google.dev/gemini-api/docs>
- [OR] OpenRouter, *OpenRouter API Reference*, 2024.
<https://openrouter.ai/docs/api-reference/overview>
- [MIS] Mistral AI, *Mistral API Documentation*, 2024.
<https://docs.mistral.ai/api/>
- [OPENAI] OpenAI, *DALL-E image generation documentation*, 2024.
<https://platform.openai.com/docs/guides/images>

9 Webographie

- [NODEJS] Node.js Foundation, *HTTP module — Node.js v20 Documentation*, 2024.
<https://nodejs.org/docs/latest-v20.x/api/http.html>
- [TUTONODEJS] MDN Web Docs, *Node.js server without a framework*, Mozilla, 2024.
https://developer.mozilla.org/en-US/docs/Learn/Server-side/Node_server_without_framework
- [MDNASYNC] MDN Web Docs, *Asynchronous JavaScript — Promises, async/await*, Mozilla, 2024.
<https://developer.mozilla.org/fr/docs/Learn/JavaScript/Asynchronous/Promises>
- [APIREST] Red Hat, *What is a REST API?*, 2024.
<https://www.redhat.com/fr/topics/api/what-is-a-rest-api>
- [APISEC] OWASP, *API Security Top 10 — Broken Object Level Authorization*, 2023.
<https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>
- [TUTOAPI] Anthropic, *Quickstart — Build with Claude*, 2025.
<https://docs.anthropic.com/en/docs/quickstart>
- [SPA] Ayooluwa Isaiah, *How to Build a Single-Page Application Without a Framework*, 2023.
<https://www.sitepoint.com/single-page-app-without-framework/>
- [ENV] The Twelve-Factor App, *III. Config — Store config in the environment*, 2017.
<https://12factor.net/config>
- [BAT] Microsoft, *Using batch files — Windows Command Reference*, 2024.
<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands>
- [MDN] Mozilla Developer Network, *JavaScript Reference*, 2024.
<https://developer.mozilla.org/fr/docs/Web/JavaScript>
- [LOCALSTORAGE] MDN Web Docs, *Window.localStorage*, Mozilla, 2024.
<https://developer.mozilla.org/fr/docs/Web/API/Window/localStorage>
- [PROMPT] Anthropic, *Prompt Engineering Overview*, 2025.
<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
- [CORS] MDN Web Docs, *Cross-Origin Resource Sharing (CORS)*, Mozilla, 2024.
<https://developer.mozilla.org/fr/docs/Web/HTTP/CORS>

10 Annexes

Annexe A : Cahier des charges

Contexte et objectif

L'objectif du projet était de réaliser une application de programmation assistée par IA dans le cadre du cours de première année de licence MIASHS. Nous avons choisi de développer un jeu de débat narratif, MindChallenger, où une IA joue le rôle d'adversaire, de juge et de coach.

Fonctionnalités demandées

- Une application fonctionnelle intégrant une IA générative via une API.
- Une interface utilisateur accessible depuis un navigateur web.
- Un code source hébergé sur un dépôt GitHub public.
- Un rapport documentant le projet, les choix techniques et le travail réalisé.

Fonctionnalités que nous avons définies

- Trois modes de jeu : débat libre, sujet surprise, mode Histoire narratif.
- Quatre adversaires IA avec des personnalités distinctes et quatre niveaux de difficulté.
- Un système de verdict et de fiche de progression personnalisée générés par l'IA.
- Un univers narratif original avec un fil scénaristique déblocable (le Carnet de Vesper).
- Une sauvegarde automatique de la progression en `localStorage`.
- Un lancement simplifié sous Windows via un script `.bat`.
- Aucune dépendance npm externe : projet autonome, lancé avec `node server.js`.

Contraintes techniques

- Langage : JavaScript (Node.js côté serveur, vanilla JS côté client).
- API utilisée : Anthropic Claude (`claude-sonnet-4-5`).
- La clé API ne doit jamais être exposée côté navigateur.
- Le projet doit fonctionner en local sans installation complexe.

Annexe A.1 : Architecture complète du projet

```
MindChallenger/
+-- server.js           # Serveur HTTP + proxy API Claude (~400 lignes)
+-- package.json        # Metadonnées du projet (zero dependance npm)
+-- .env.example        # Modele de configuration (a copier en .env.local)
+-- .env.local          # Cle API et config (non commitee, dans .gitignore)
+-- .gitignore          # Exclusions Git
+-- Lancer-MindChallenger.bat # Script de lancement Windows en un clic
```

```

+-- DEMARRAGE.txt          # Guide rapide 3 etapes (utilisateur non-dev)
+-- README.md              # Documentation complete du projet
+-- public/
|   +-- index.html         # Page HTML unique (point d'entree)
|   +-- src/
|       +-- app.js         # Toute la logique frontend (~1339 lignes)
|       +-- styles.css     # Design complet (~1362 lignes)
|   +-- assets/
|       +-- characters/    # Portraits PNG des adversaires et de la juge
|       +-- players/       # Avatars PNG des 3 joueurs
|       +-- backgrounds/   # Fonds de scene par chapitre (8 images)
+-- docs/
    +-- API_SETUP.md       # Guide d'obtention et de configuration de la cle API
    +-- ARCHITECTURE.md    # Description de chaque fichier du projet
    +-- PROMPTS_IA.md      # Documentation des prompts et comportements IA

```

Annexe B : Exemple d'exécution du projet

Les captures d'écran suivantes illustrent une session de jeu complète, du prologue jusqu'au Carnet de Vesper.



FIGURE 1 – Prologue — Le train vers Aster, introduction narrative au mode Histoire



FIGURE 2 – Sélection du protagoniste parmi les trois avatars disponibles



FIGURE 3 – Barre de navigation et ticker de citations humoristiques

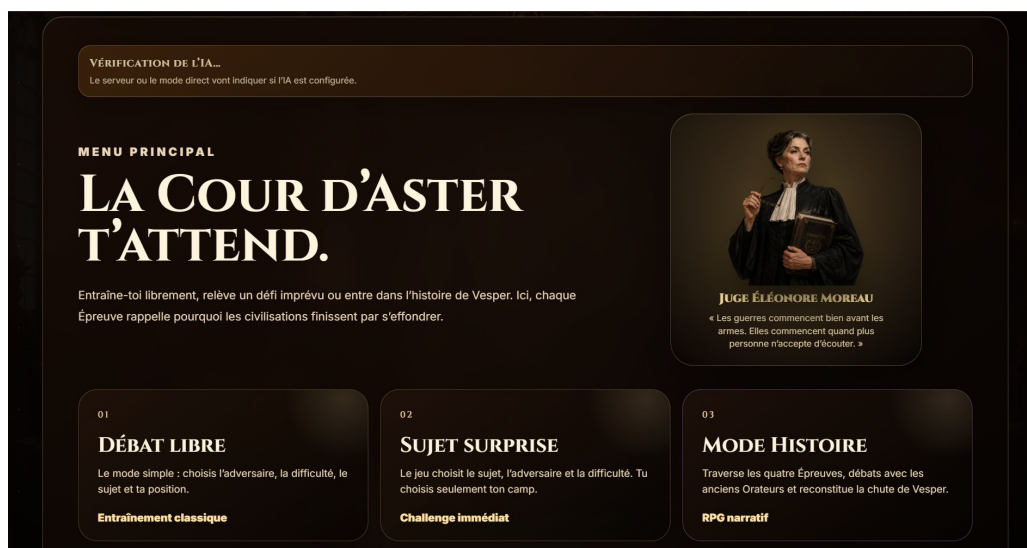


FIGURE 4 – Menu principal — présentation des trois modes de jeu

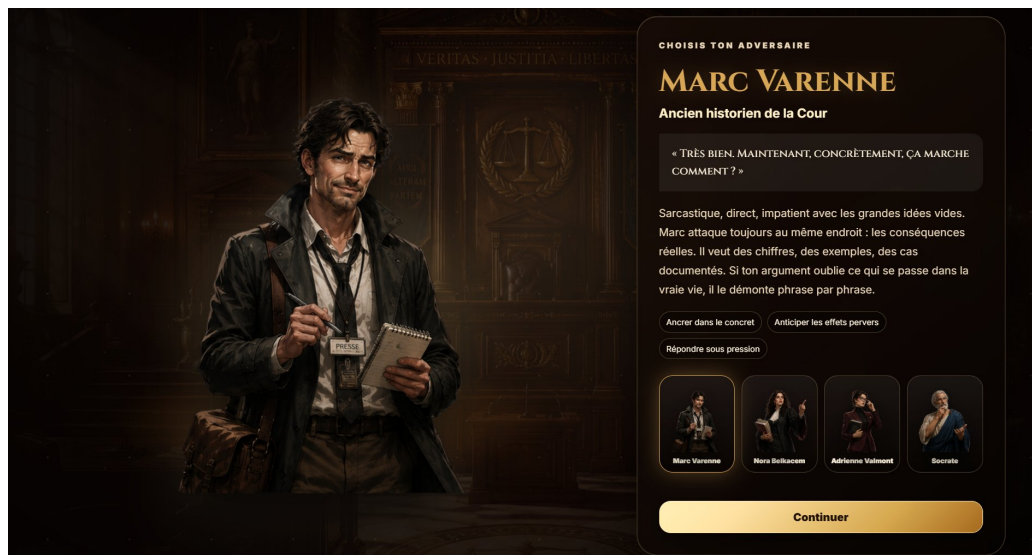


FIGURE 5 – Sélection de l'adversaire — fiche de Marc Varenne

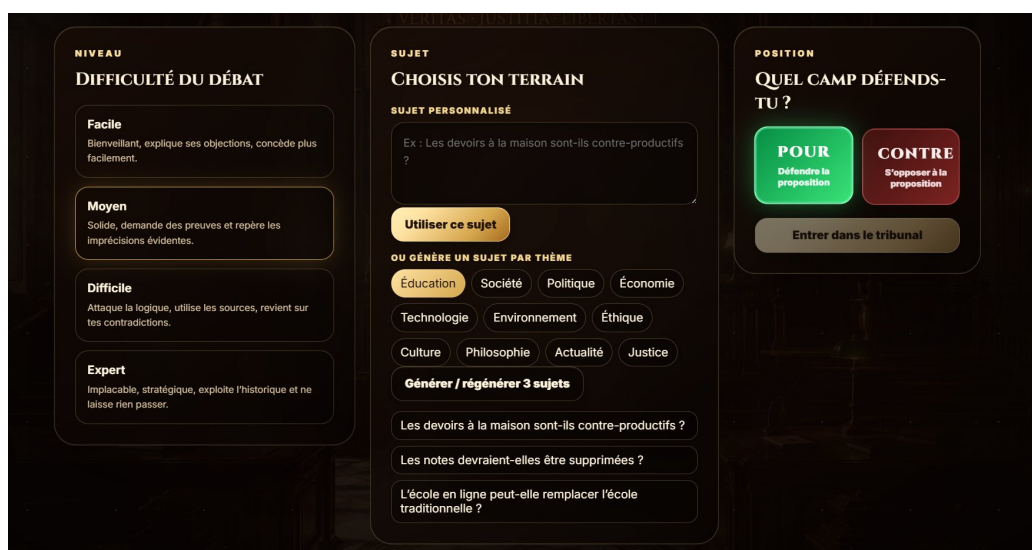


FIGURE 6 – Configuration d'un débat libre : niveau, sujet et camp



FIGURE 7 – Mode Sujet surprise — Socrate en adversaire, sujet généré par l'IA



FIGURE 8 – Interface de débat avec Nora Belkacem — premier échange



FIGURE 9 – Mode Histoire — introduction du Chapitre III avec Adrienne Valmont

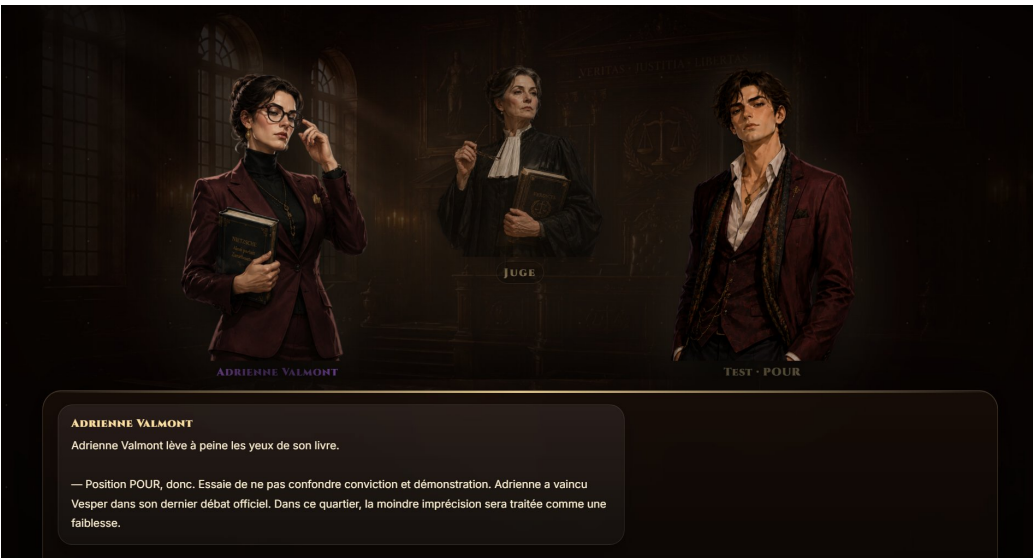


FIGURE 10 – Débat en cours avec Adrienne Valmont



FIGURE 11 – Épilogue — Le Carnet de Vesper, accessible après le mode Histoire



FIGURE 12 – Le Carnet de Vesper — pages 1 et 2



FIGURE 13 – Dernières pages du Carnet — la page 6 contient l’easter egg

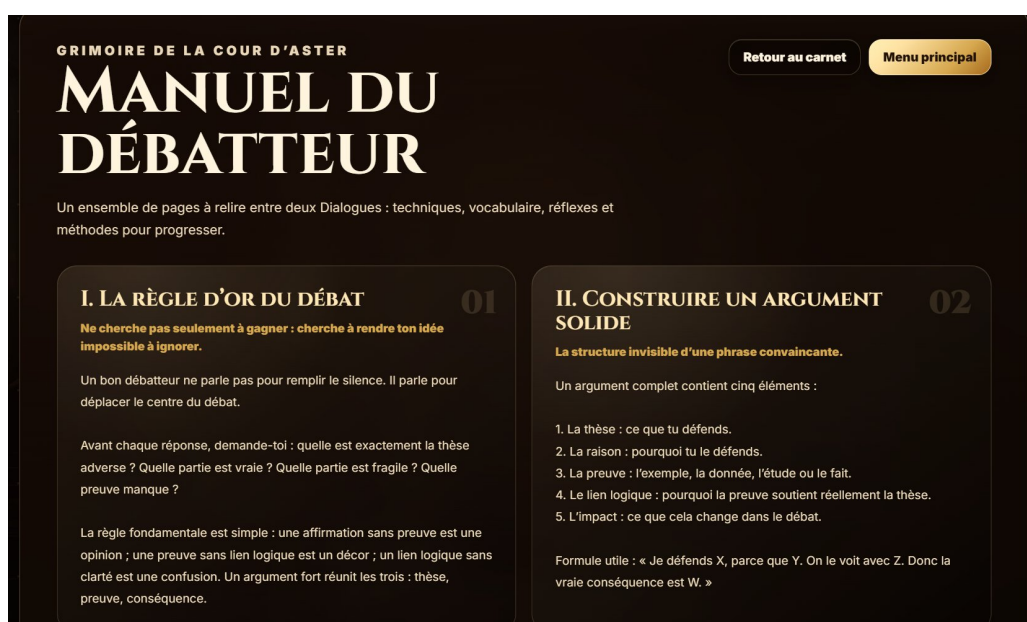


FIGURE 14 – Le Grimoire — Manuel du débatteur, pages I et II

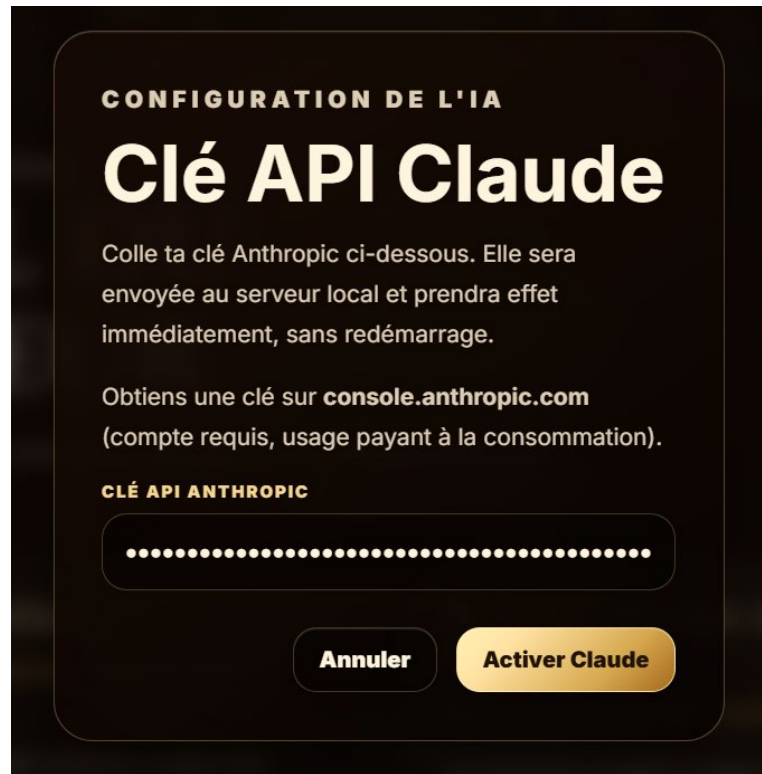


FIGURE 15 – Interface de configuration de la clé API — saisie sans redémarrage

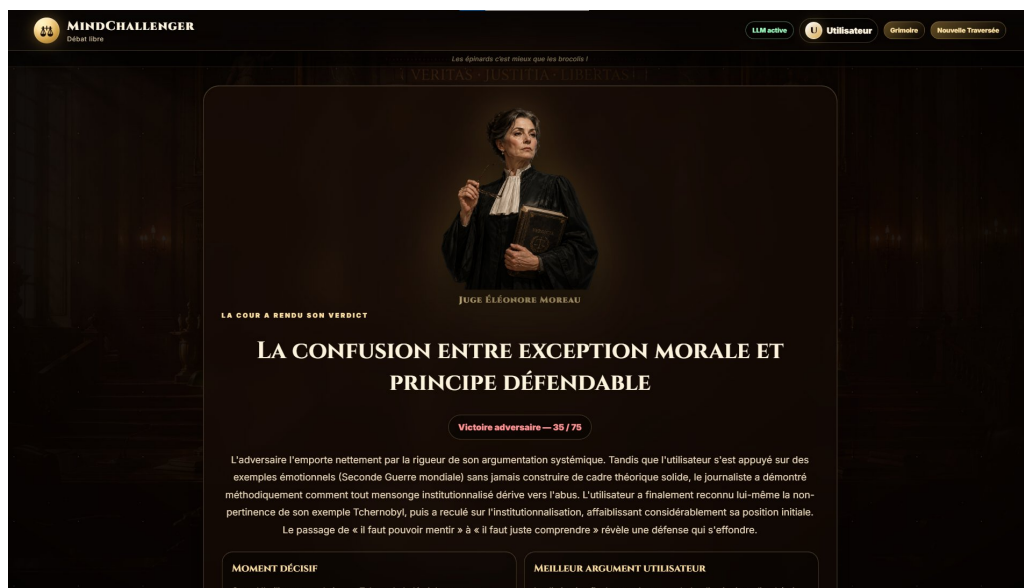


FIGURE 16 – Écran de verdict — analyse de la juge Moreau avec score et moment décisif

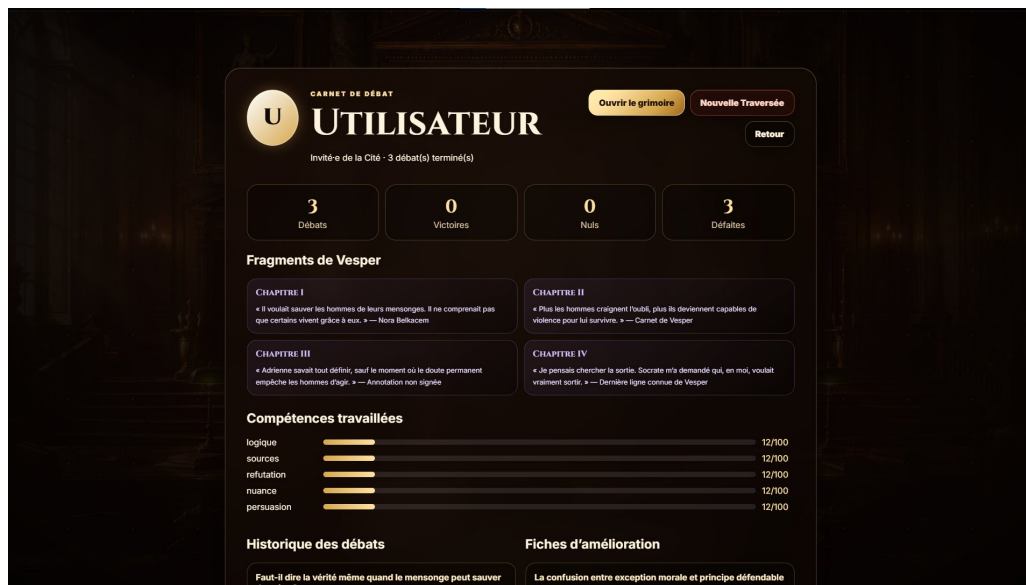


FIGURE 17 – Carnet de débat — statistiques, compétences, Fragments de Vesper et historique



FIGURE 18 – Première étape de réinitialisation — confirmation avant de repartir du prologue



FIGURE 19 – Deuxième étape — confirmation définitive d’effacement de la progression

Annexe C : Manuel utilisateur

Prérequis

- Node.js version 20 ou supérieure : <https://nodejs.org>
- Une clé API Anthropic (commence par `sk-ant-`) : <https://console.anthropic.com>

Clé API pour l’évaluation : la clé API Anthropic n’est pas incluse dans ce document ni sur le dépôt GitHub (le fichier `.env.local` est exclu par `.gitignore`). Elle peut être saisie directement dans l’interface via le bouton *Vérification IA* en haut à droite, sans redémarrage du serveur.

Lancement sous Windows (méthode recommandée)

Double-cliquer sur `Lancer-MindChallenger.bat`. Le script gère tout automatiquement et ouvre le navigateur sur `http://localhost:5173`.

Lancement manuel (tous systèmes)

```
1 cd MindChallenger
2 cp .env.example .env.local
3 # Éditer .env.local : ANTHROPIC_API_KEY=sk-ant-votre-cle-ici
4 node server.js
5 # Ouvrir http://localhost:5173 dans le navigateur
```

Navigation dans le jeu

1. Choisir son avatar et entrer son prénom.
2. Depuis le menu principal, choisir un mode : *Débat libre*, *Surprise* ou *Histoire*.

3. En mode Débat libre : sélectionner un adversaire, saisir un sujet, choisir son camp et la difficulté.
4. Taper ses arguments et les envoyer.
5. Après au moins 2 échanges, cliquer sur *Verdict* pour obtenir le jugement et la fiche de progression.
6. Le Carnet de Vesper (et son *easter egg* final) se déverrouille uniquement après avoir complété l'intégralité du mode Histoire, épilogue inclus.